

# VeriBeca

## Informe del proyecto

---

Asignación transparente de becas con DSL tipado, IA y Blockchain

Teoría de Computación 2 · Universidad Champagnat

Equipo de 3 integrantes

Repositorio: [github.com/santivicente/veribeca](https://github.com/santivicente/veribeca)

Video demo: [enlace en Google Drive](#)

## 1. Resumen ejecutivo

**VeriBeca** es una plataforma que hace que la asignación de becas y ayudas sociales sea **justa y auditable**. La institución define sus criterios de elegibilidad en un lenguaje propio y tipado (un DSL); el sistema los valida y los aplica automáticamente a todos los postulantes por igual; una inteligencia artificial prioriza a los elegibles según su vulnerabilidad; y cada decisión queda registrada de forma inmutable en blockchain como un certificado de auditoría verificable por cualquiera.

El proyecto integra de forma real las cuatro unidades de la materia —lenguajes, sistemas de tipos, compiladores y seguridad/blockchain— más Inteligencia Artificial, en un único flujo.

## 2. Problema y oportunidad

La asignación de becas, subsidios y ayudas sociales en universidades, municipios y ONGs se percibe como poco transparente: criterios ambiguos, evaluación manual y lenta, y sospechas de favoritismo. Cuando un postulante queda afuera, no puede auditar por qué, ni hay garantía de que la regla aplicada haya sido la misma para todos.

**Usuarios afectados:** áreas de bienestar estudiantil, municipios y ONGs (que deben decidir de forma justa y defendible), y los postulantes (que reciben decisiones hoy difíciles de verificar). **Relevancia:** son fondos significativos que afectan la continuidad educativa de poblaciones vulnerables; la opacidad erosiona la confianza institucional.

## 3. Solución propuesta

VeriBeca resuelve el problema en cinco pasos:

- **Escribir la regla** en un lenguaje simple y legible (el DSL).
- **Validar** la regla con un type-checker, antes de aplicarla.
- **Aplicarla** automáticamente a todos los postulantes (el intérprete).
- **Priorizar** a los elegibles por vulnerabilidad (la IA).
- **Sellar** cada decisión en blockchain (hash en un smart contract).

**Diferencia frente a alternativas:** una planilla o un sistema cerrado no validan las reglas formalmente ni dejan evidencia inalterable. VeriBeca combina un lenguaje verificable con un registro inmutable, algo que esas soluciones no ofrecen.

## 4. MVP desarrollado

Se implementó el flujo completo de punta a punta:

- Motor DSL 100% funcional y testeado: lexer, parser (AST), type-checker e intérprete.
- Backend FastAPI con endpoints para cargar reglas, cargar postulantes y evaluar.
- Modelo de IA (scikit-learn) de scoring/priorización sobre dataset sintético.
- Smart contract en Solidity desplegado en testnet + cliente web3 con fallback local.
- Frontend Streamlit que integra todo el flujo.

**Qué funciona hoy:** todo el pipeline (escribir y validar la regla, evaluar, priorizar y generar el certificado on-chain o en respaldo). **Pendiente para futuras versiones:** autenticación de usuarios, multi-institución, base de datos y despliegue productivo.

## 5. Componentes tecnológicos

### Arquitectura

```
[Streamlit UI] --> [FastAPI backend]
                        |- Motor DSL: lexer -> parser -> type-checker -> interprete
                        |- IA (scikit-learn): scoring de prioridad
                        |- Web3: hash(decision) -> Smart Contract (Sepolia) [+ fallback]
```

### Inteligencia Artificial

Se usó **scikit-learn**. Se eligió por su **simplicidad e interpretabilidad**: en decisiones sensibles es más importante poder explicar por qué alguien tiene mayor prioridad que usar un modelo opaco. Aporta un orden de prioridad consistente y explicable, basado en indicadores de vulnerabilidad (ingreso, integrantes de la familia, distancia).

### Blockchain

Se desplegó un smart contract **AuditoriaBecas** (Solidity) en la testnet **Ethereum Sepolia**. Por cada decisión se calcula un hash (SHA-256 de datos + regla + resultado) y se registra con web3.py. Resuelve la **auditabilidad**: la decisión no se puede alterar y cualquiera la verifica en el explorer.

### Mecanismo de fallback (tolerancia a fallos)

El registro on-chain está envuelto en un bloque **try / except**. Si el registro en la red falla por cualquier motivo (sin conexión, RPC caído, falta de configuración o de fondos), se captura la excepción y se guarda **el mismo hash** en un **ledger local** (archivo JSON), marcando la decisión como modo *fallback*. Como el hash es idéntico al que iría on-chain, la integridad de la auditoría se mantiene y **el sistema nunca se interrumpe**; al haber conexión, la decisión se registra directamente en la blockchain.

### Integración con la materia (>=3 líneas)

| Línea temática            | Aplicación concreta                                    |
|---------------------------|--------------------------------------------------------|
| Lenguajes de programación | DSL propio de reglas de elegibilidad                   |
| Sistemas de tipos         | Type-checker que valida las reglas antes de aplicarlas |
| Diseño de compiladores    | Pipeline lexer -> parser -> AST -> interprete          |
| Seguridad / Blockchain    | Hash de cada decision + smart contract en testnet      |

## 6. Innovación e impacto

**Innovación:** combinar un lenguaje verificable de reglas (con validación de tipos) con un registro inmutable y priorización por IA, aplicado a un problema social concreto. No es 'blockchain por moda': el hash on-chain resuelve específicamente la auditabilidad.

### Impacto esperado:

- Social: más confianza en las instituciones y menos discrecionalidad.
- Económico: reducción drástica del tiempo de evaluación frente al proceso manual.
- Ambiental: uso de una testnet PoS de bajo consumo energético (Ethereum Sepolia).
- Cómo medir el éxito: tiempo de evaluación, % de decisiones con certificado verificable, y reducción de reclamos sin evidencia.

## 7. Modelo de negocio

### A quién apuntamos (segmentos de clientes)

- **Universidades** — áreas de bienestar estudiantil (cliente inicial / piloto).
- **Municipios** que otorgan subsidios y becas.
- **ONGs y fundaciones** que distribuyen ayudas (becas, alimentos, microcréditos).

El **usuario final** son los postulantes, que reciben una decisión justa y verificable.

### Propuesta de valor

Decisiones de asignación **justas, consistentes y auditables por cualquiera**: reglas tipadas que evitan errores antes de afectar a personas, aplicación igual para todos, y trazabilidad inmutable que protege la reputación de la institución ante reclamos o auditorías.

### Cómo se monetiza (quién paga)

- **Suscripción SaaS** por institución (licencia anual).
- **Servicios** de implementación, integración y capacitación.
- Modelo **open-core**: versión base gratuita + módulos premium (reportería, integraciones).

**Pagan las instituciones** que asignan las ayudas: ganan eficiencia, capacidad de defender sus decisiones ante auditorías y mayor confianza de la comunidad.

### Canales y crecimiento

- Convenios directos con áreas de bienestar universitario y municipios.
- Demos en jornadas de gestión pública y transparencia.
- Repositorio open-source como vía de adopción y prueba inicial.
- Estrategia: arrancar con una universidad piloto y expandir a municipios y ONGs.

## 8. Viabilidad y sostenibilidad

- **Técnica**: demostrada — el MVP funciona end-to-end, con 27 tests y una transacción real en testnet.
- **Económica**: costos bajos (testnet gratuita; en producción, una L2 de gas mínimo).
- **Operativa**: la institución solo escribe reglas y carga postulantes.

- **Aliados:** bienestar universitario, municipios, ONGs y fundaciones de transparencia.

## 9. Trabajo en equipo

Equipo de 3 integrantes, con metodología ágil: backlog por épicas (Definición -> MVP -> Demoday), módulos desacoplados desarrollados en paralelo y commits frecuentes.

| Rol                                  | Responsabilidades                                 |
|--------------------------------------|---------------------------------------------------|
| Lenguajes, compiladores y tipos      | DSL: lexer, parser, AST, type-checker, interprete |
| IA y backend                         | Dataset, modelo de scoring, API FastAPI           |
| Blockchain, frontend y documentacion | Smart contract, web3, UI Streamlit, docs          |

## 10. Métricas, evidencia y estrategia de pitch

- 27 pruebas automáticas en verde (DSL 20 · IA 3 · backend 2 · contracts 2).
- Sobre 100 postulantes de prueba: 24 elegibles, priorizados en segundos.
- Smart contract desplegado y transacción real verificable en Ethereum Sepolia.
- Repositorio público y ordenado con toda la documentación.

**Comunicación y pitch:** presentación de 7 minutos + 3 de preguntas, repartida entre 2 expositores siguiendo las 10 slides. Hilo: problema -> solución -> demo -> cómo cubre la materia -> impacto -> negocio -> cierre. Mensaje central: *"De 'confíen en nosotros' a 'verifíquenlo ustedes mismos'"*. Poco texto por slide, demo en vivo para evidenciar que funciona, y respuestas preparadas para las preguntas del jurado.